

Software Requirements Specification (SRS)

Project Name: EnergiCast

Version: 1.0

1. Introduction

1.1 Purpose

The purpose of this document is to define the Software Requirements Specification (SRS) for EnergiCast, a role-based web application designed for energy consumption forecasting using time-series machine learning models.

1.2 Scope

EnergiCast provides a platform where administrators manage controlled historical energy datasets and publish validated forecasting models. Authenticated users can then select available energy types, supported features, and forecast horizons to request predictions.

- **In Scope:** Admin/User authentication, dataset management, time-series exploratory data analysis (EDA), feature engineering, baseline and machine learning modeling, model evaluation/publication, and user forecasting visualization.
- **Out of Scope (Initial Release):** Real-time IoT or smart-meter streaming, Kafka/MQTT streaming infrastructure, unrestricted user-provided datasets, large-scale distributed training, and automatic retraining.

1.3 References

- EnergiCast_Final_Product_Documentation_v2.pdf (Primary requirement source)

2. Overall Description

2.1 Product Perspective

EnergiCast operates as a split-architecture web application. It utilizes a React-based frontend, a FastAPI (Python) backend, and PostgreSQL for persistent data storage. Machine learning pipelines are integrated into the backend using standard Python data science libraries.

2.2 User Roles and Characteristics

1. **Administrator:** Responsible for system management, data integrity, and model validity. Capable of uploading datasets, configuring features, training models, and publishing final models for user access.
2. **User:** An authenticated end-user who utilizes the published models. They select energy

types, supported features, and forecast horizons to generate and view future predictions.

2.3 Operating Environment

- **Frontend:** React, Vite, Tailwind CSS (Hosted on platforms like Vercel).
- **Backend:** FastAPI, Python (Hosted on platforms like Render).
- **Database:** PostgreSQL (Managed, e.g., Neon).
- **Data/Model Artifacts:** Joblib/file storage for serialized models and CSV datasets.

3. System Features

3.1 Administrative Capabilities

- **Dataset Management:** Upload, validate, replace, delete, and publish CSV datasets. The system must parse timestamps, detect duplicates, and report missing observations.
- **Feature Configuration:** Define available exogenous features (e.g., temperature) and forecast horizons for specific energy types.
- **Model Training & Evaluation:** Train models using a chronological split strategy (Train/Validation/Test). Compare baseline (Naive, ARIMA) and Feature-based ML (Random Forest, XGBoost) models using MAE, RMSE, MAPE.
- **Model Publication:** Select the best-performing model and publish it for user availability.

3.2 User Capabilities

- **Authentication:** Secure registration and login functionalities.
- **Forecasting Requests:** Select a published energy type, configure supported features, and input a forecast horizon.
- **Visualization:** View graphical charts comparing historical consumption versus predicted future consumption.
- **History:** Access a persistent history of previously requested forecasts.

3.3 Machine Learning Pipeline

- **Preprocessing:** Chronological sorting, timestamp normalization, missing value handling, and outlier detection.
- **Feature Engineering:** Generation of calendar features (hour, day, season), lag features (lag-1, lag-24), rolling means/std deviations, and exogenous variable integration.
- **Forecasting Methods:** Recursive prediction, direct multi-output prediction, or model-specific APIs.

4. External Interface Requirements

4.1 User Interfaces

- **Login/Register Pages:** Standard form-based authentication.
- **Admin Dashboard:** Interfaces for Dataset Management (FileUpload, DatasetCard), Feature Management, and Model Comparison.
- **User Dashboard:** Forecasting interface (EnergySelector, FeatureSelector,

ForecastHorizon) and Results views (MetricsCard, ForecastChart).

4.2 Software Interfaces (API REST Endpoints)

- POST /api/auth/register, POST /api/auth/login - Authentication.
- GET /api/datasets, POST /api/admin/datasets, PUT /api/admin/datasets/{id} - Dataset lifecycle.
- POST /api/admin/models/train, GET /api/admin/models/{dataset_id} - Model training and metrics.
- POST /api/forecast, GET /api/forecasts - Forecast generation and history.

5. Non-Functional Requirements

5.1 Security

- **Authentication:** JWT-based sessions with a strong, non-committed secret.
- **Passwords:** Secure cryptographic hashing; no plaintext storage.
- **Authorization:** Strict Role-Based Access Control (RBAC) enforced on the backend API, not just the frontend UX.
- **Validation:** Strict validation of uploaded file types, sizes, and structure to prevent path traversal and malicious payloads.

5.2 Performance & Reliability

- **Compute Isolation:** Model training is computationally expensive; initial systems handle this synchronously via Admin, but timeouts must be managed (background workers are scoped for the future).
- **Data Integrity:** The system relies on rigid database schemas in PostgreSQL (Users, Datasets, Features, Models, Forecasts, Forecast Values) to maintain relational integrity.

5.3 Maintainability

- The ML pipeline must remain modular so that new time-series models (like deep learning architectures) or energy types can be seamlessly added in future iterations.